

02-719 Week 2

Genomics and Epigenetics of the Brain

Aidan Jan

August 31, 2026

Basic Experiment Setup

Suppose we are running an experiment with 3 different animals, in two conditions. Let A , B , C be in condition A and D , E , F be in condition B. Let a scale from 0 to 10 define the amount of open chromatin. Suppose we run our experiment, and we get:

Species x Condition	Open Chromatin
A	2
B	4
C	3
D	7
E	10
F	8

From this, we can then compare the effects on the two different conditions on epigenetics. Some statistics we can use to quantify the difference are

- Mean and Std. Dev
- t-test
- Linear Regression
- etc.

Now, if today we want to understand the conditions' effects on multiple genes, we can create matrix where there are g rows, where g is the number of genes we care about, and c number of columns, where c is the number of species x conditions. Data can then be compared and processed efficiently using a computer program.

Introduction to R Programming Language

- Very similar to Matlab
- Handles matrix operations very well
 - For loops are quite slow
- R is open source
 - The underlying code is available
 - No license required
- Many people have put together statistical and genomic packages in for R
 - [Bioconductor](#) - Open source software for bioinformatics
 - Limma - linear models for microarray (and RNA-Seq)

Data / Class Types in R

- Boolean, Integer, Numeric (double), String, Factor
- Vector - 1D, every element same data type
- List - 1D, every element can be different type
- Matrix - 2D, every element same data type
- DataFrame - 2D, every column same data type
- Factor - Used for representing qualitative variables in statistical operations

Assigning Different Types

- `x <- 1`: assignment operator
- `class(x)`: identify data type with class function
- `x <- c(1, 2, 3)`: assign a vector
- `x <- matrix(c(1, 2, 3, 4, 5, 6), 2, 3)`: assign a matrix. This example makes a matrix of 3 rows, 2 cols. Numbers fill rows first (1, 2, 3) in col 1.
- `f <- data.frame(num=c(1, 2, 3, 4), letter=c("a", "b", "c", "d"))`: sets data in a data frame. column name is the key.
- `rownames(f) <- c("first", "second", "third", "fourth")`: sets row names for a dataframe

Gene Ontology

- When we perform a ChIP-Seq experiment, have peaks (signal) that can potentially cover any part of the entire genome, not just genes.
- The GREAT gene ontology analysis tool is designed to handle this case.

Expected Counts

Problem: We have a genome that contains 100 genes.

- 10 of those genes are involved in DNA replication
- We conduct an experiment where key replication gene is knocked out
- As a result, 20 genes change in expression
- 8 of those genes are involved in replication
- What is the significance?

To solve: Let n be number of genes (20), k be number of genes in category (8), and p be probability that any given gene is in the category: ($10/100 = 0.1$).

By simple probability, we would expect that ($20 \text{ genes} \times 0.1 \text{ probability} =$) 2 genes are in the category. What we observe was 8 genes, which is much higher than the predicted 2 genes. $P_{\text{binom}} = 0.0004$.

Binomial P-value

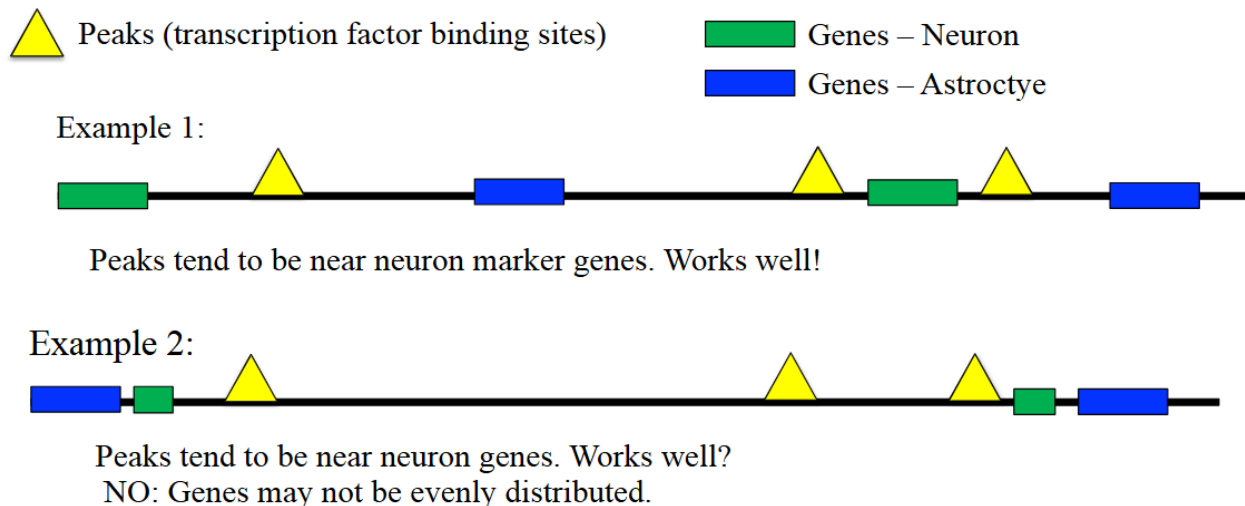
- We can do statistical enrichments based on groups of genes:
 - How many genes do I see in a category?
 - How many genes would I expect by chance?

$$\Pr(k; n, p) = \binom{n}{k} \cdot p^k \cdot (1-p)^{n-k}$$

- In the formula above,
 - n = number of trials (number of genes)
 - k = number of successes (number of genes in category)
 - p = probability of success (probability that any given gene is in the category)

Mapping Regulatory Elements to Genes

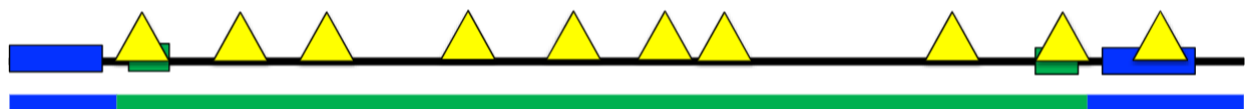
The assumptions of the binomial test may not be valid.



GREAT

Solution: Partition the genome based on the gene ontology category of the nearest gene. Then, adjust the probability of a hit based on the proportion of the genome that is covered, not the total number of genes in the category.

Example 2:



Original Test: P-value = $\text{binom}(n=10, k=9, p=0.5) = 0.01$

New Test: P-value = $\text{binom}(n=10, k=9, p=0.9) = 0.73$